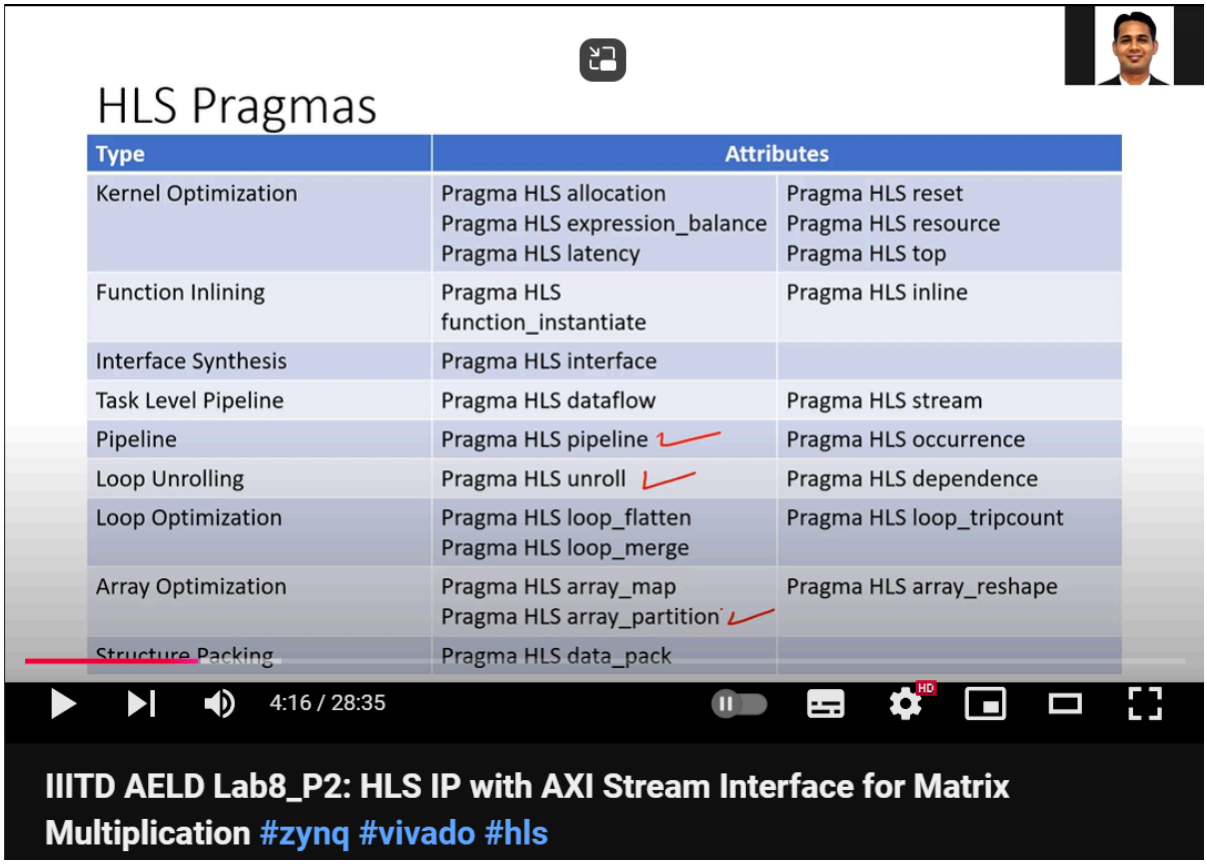
Vivado HLS 2019.1 - HLS Pragma Optimizations

Most pictures here were extracted from the video: IIITD AELD Lab8_P2.

Introduction

It is worth mentioning that the more resources used will reduce execution time but it will increase device cost and energy consomption. Main `pragmas`:



Only some `pragmas` will be explained here.

HLS Pragmas

Pipeline

pipeline allows operations to happen concurrently, meaning that it is not necessary to wait until all steps of an operation are finished before starting the steps of the next operation. pipeline allows better use of the hardware without expanding it.

HLS Pipeline

- Loop pipelining can be **prevented** by loop carry dependencies.
- All loops in the hierarchy below are automatically unrolled
- Any function in the hierarchy below must be pipelined individually. Otherwise, it may affect the performance

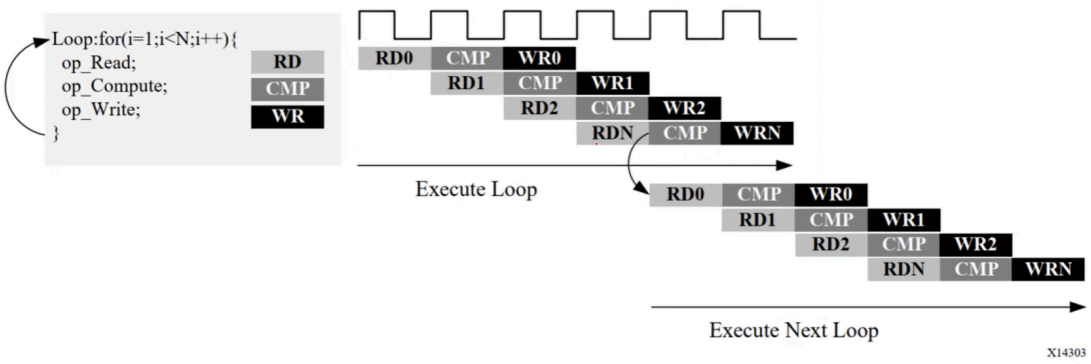
```
#pragma HLS pipeline II=<int> enable_flush rewind
```

6:42 / 24:16

HLS Pipeline

```
#pragma HLS pipeline II=<int> enable_flush
```

- **Rewind:** Enables the overlap of the iterations of successive calls to the rewind loop

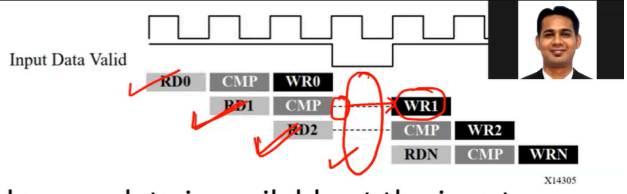


Lab8_P3: HLS IP with AXI Stream Interface for Matrix Multiplication #zynq #vivado #hls

hms to Architecture, ECE, IIIT Delhi

All | From the series | From Algorithms

HLS Pipeline



- Pipelines continue to execute as long as data is available at the input of the pipeline. If there is no data available to process, the pipeline will stall.
- In some cases, it is desirable to have a pipeline that can be "emptied" or "flushed".
- When a pipeline is "flushed" the pipeline stops reading new inputs when none are available (as determined by a data valid signal at the start of the pipeline) but continues processing, shutting down each successive pipeline stage, until the final input has been processed through to the output of the pipeline.

```
#pragma HLS pipeline II=<int> enable_flush rewind
```

10:47 / 24:16

Array Partition

HLS array_Partition

- Important for optimal pipelining due to limited number of read/write ports on memory
- Arrays are implemented as block RAM (memory unit inside FPGA) which has maximum two ports per unit.
- For multiple simultaneous read/write, array needs to be split up into small BRAM units (i.e. more ports)